Topology-Aware Mapping for Photonic Interconnects in TPU-Class LLM Accelerators

Pranav Sitaraman • Simone Surdi

MIT 6.5930 / Hardware Architectures for Deep Learning • Spring 2026 • Project 12: Evaluate a New Technology

1. Motivation

Why photonic interconnect for TPU pods? Modern LLM systems are limited not by matrix-multiply throughput but by the cost of moving activations, KV-cache state, expert tokens, gradients, and shared weights between chips. TPU v5p and Ironwood/TPU7x are organised around high-radix ICI exactly for this reason. Co-packaged photonics changes *four* interconnect properties at once:

- Energy/bit drops 3–5× (10 pJ/bit → 3.2/1.9 pJ/bit).
- Bandwidth grows 4–70× (0.8 Tb/s → 57.3 Tb/s).
- Topology can become a flat switched fabric, not a multi-hop torus.
- Hardware multicast replaces N unicasts.

Research question. Is photonic interconnect a constant-factor link substitution, or does it change *how a TPU pod should be mapped and scheduled* for LLM workloads? Following 6.5930's mapping formalism (partitioning + dataflow + tile sizing), we separate intra-chip mapping from cluster-level distributed schedule and ask which layer the gain actually lives in.

2. Underlying Principles

Modeling stack: four layers.

1. **Workload (einsum).** `single_matmul.yaml`: $O_{m,n} = \Sigma_k A_{m,k} W_{k,n}$ with rank sizes per LLM stage.
2. **Architecture.** `tpu_lightmatter_cluster.yaml`: `LogicalClusterMemory` → `NodeArray` fanout → `HBM` → `UnifiedBuffer` → `ProcessingElement` → `Register/MAC`.
3. **Mapping.** AccelForge mapper output: `!Storage/!Temporal/!Spatial/!Compute` sequence – the 6.5930 mapping formalism. Embodies the intra-PE dataflow.
4. **Distributed schedule.** `COMM_PATTERN`, the topology cost model, and four named XLA/GSPMD policies: `spmd_collective`, `remote_hbm_streaming`, `replicated_hbm_cache`, `photonic_multicast_cache`.

Network cost model. For traffic of B bits, hop count h, link energy e_ℓ, router latency t_r: $E_{\text{net}} = B\,h\,e_\ell$, $T_{\text{net}} = B/(BW_\ell/C(N)) + h\,t_r$. $C(N)$: $\sqrt{N}$ mesh, $\sqrt[3]{N}$ torus, 1 crossbar. Multicast: non-MC pays N unicasts; MC pays unique payload once.

Iterative feedback loop. Run mapper → extract `LogicalClusterMemory` traffic → price via topology → fold network cost back into logical-memory energy/latency → re-run until stable. Converges in ≤ 3 iterations.

Superpod stage model. $T_{\text{stage}} = \max(T_c, T_{\text{HBM}}) + (1-\alpha)T_{\text{net}}$, $E_{\text{stage}} = E_c + E_{\text{HBM}} + E_{\text{net}}$.

Modeled technologies

Tech	BW	pJ/bit
Electrical 800G	0.8	10.0
Adv. chiplet	3.5	6.0
EVK50	0.8	4.6
EVK100	3.2	**3.2/1.9**
L200	16	4.5
L200X	32	4.8
M1000	57.3	**2.3**

EVK100: conservative public target / Passage-family sensitivity. M1000: 57.3 Tb/s one-way from 114.6 Tb/s bidirectional.

Workloads: 6 GEMM proxies for LLM stages + Flash (120B dense), Pro MoE (4.7T/600B/32K), Deep Think (4.7T/750B/131K).

What we hold fixed: compute, MAC array, register, UB, HBM, HBM pJ/bit/latency. Only interconnect, topology, and schedule vary.

3. Evaluation

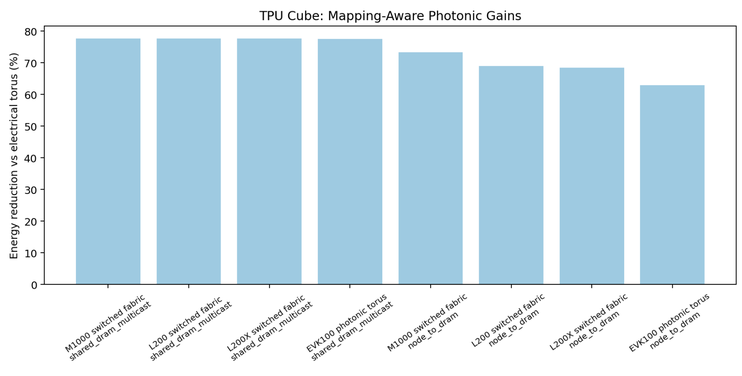
Experiments.

1. 16-node AccelForge cluster – electrical 4×4 mesh vs EVK100 optical crossbar.
2. 64-chip TPU cube – electrical 4×4×4 torus vs EVK100/L200/L200X/M1000.
3. Distributed compiler-visible schedule comparison.
4. Scaling sweep – 64 to 9216 chips.
5. Advanced chiplet baseline (3.5 Tb/s, 6 pJ/bit).
6. 9216-chip Ironwood-class superpod – Gemini Flash / Pro MoE / Deep Think proxies.
7. Ablation ladder isolating link-energy, topology, multicast, schedule, and full photonic stack.

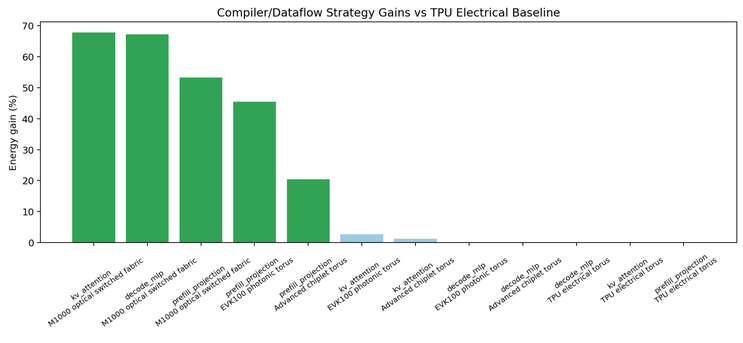
Reproducibility. `make study && make notebook && make test` in the AccelForge Docker image. 71 unit/integration tests assert the technology table, network model, mapper-loop convergence, scaling exponents, and superpod summaries.

Bias controls. (i) Multiple Lightmatter sources cross-checked; TPU values from Google Cloud docs and Hot Chips 2025. (ii) Strong chiplet baseline rules out raw bandwidth as the only explanation. (iii) Optical multicast modelled optimistically as one logical broadcast – explicit assumption. (iv) Gemini parameter counts unpublished, so we use proxy stress workloads, clearly labelled as proxies.

4. Topology / Compiler

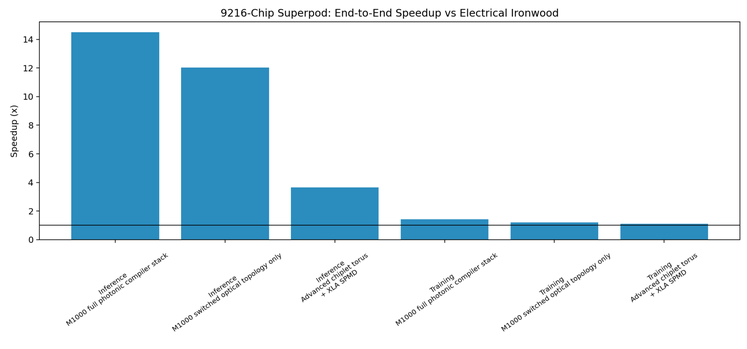


TPU cube: M1000 multicast removes 99.91% of network energy.

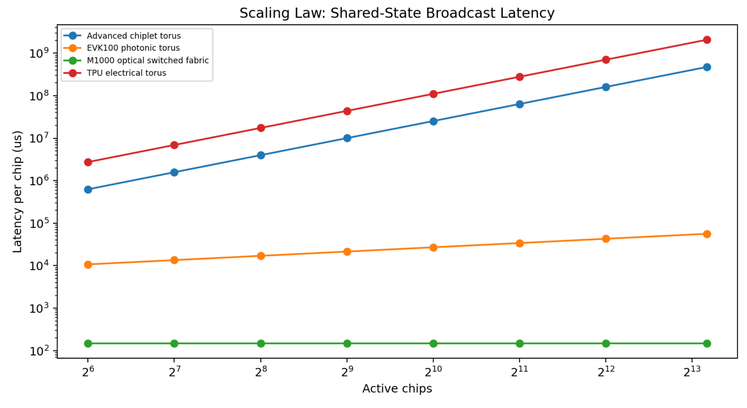


Compiler-visible schedule changes whenever multicast becomes available.

5. Superpod



Long-context inference benefits 14.50×; training only 1.43×.



Optical multicast scaling exponent stays at 0.00 across 64→9216 chips.

6. Results and Analysis of Results

Three key insights.

1. **The intra-chip mapping is stable; the distributed schedule is what changes.** Across 24 (workload, technology, scenario) cases on the TPU cube, *0/24* inner mapping signatures change. The benefit lives in layer 4: *5/16* workload–scenario pairs flip their optimal communication pattern (node-to-node → shared-DRAM multicast), and the optimal compiler-visible schedule changes whenever multicast becomes available. Reporting these as separate layers matches the 6.5930 mapping formalism cleanly. This is the central conceptual finding.
2. **Hardware multicast, not raw bandwidth, dominates shared-state workloads.** The 3.5 Tb/s, 6 pJ/bit chiplet baseline is still beaten by M1000 by **99.85%** on shared-DRAM multicast: N unicasts collapse to one logical broadcast, and contention disappears on a switched fabric. The same chiplet is competitive on node-to-node activation exchange.
3. **Photonics is not a uniform multiplier.** On the 9216-chip superpod, long-context inference reaches **14.50**× speedup; training reaches only **1.43**×. Once collectives shrink, training becomes compute-dominated; inference stays network-exposed because decode and MoE routing cross the fabric every step. The strong case is inference and long-context serving, not training payoff.

Axis	Value	Setting	Axis	Value	Setting
16-node energy red.	72.21%	EVK100 MC	Compiler best gain	67.75%	M1000 stream
16-node latency red.	97.57%	EVK100 MC	Multicast slope	0.00	M1000 64→9216
TPU cube net energy	99.91%	M1000 MC	Chiplet vs M1000	99.85%	MC
TPU cube map energy	77.70%	M1000 KV	Inference speedup	14.50×	Deep Think
Inner maps changed	0/24	cube	Training speedup	1.43×	Pro MoE
Distributed maps	5/16	cluster	Inference energy	51.07%	Deep Think

Where it helps. Long-context inference; sparse-MoE serving; pooled remote HBM; cache-fill / weight-broadcast traffic.

Where it does not help. Local nearest-neighbor exchange (electrical torus already efficient at one hop); compute-dominated training stages; mappings already optimal on electrical.

What worked / did not. Worked: feedback loop converged in ≤ 3 iterations; multicast asymmetry as predicted; superpod stage model reproduced inference-vs-training divergence. Did not work: encoding the inter-chip NoC as a degenerate `!Memory` inside AccelForge produced opaque numbers; we kept the network model external. The inner mapping signature did not change – an honest negative result we report rather than hide.

Future work. Direct XLA/Pallas integration; cycle-accurate optical-switch simulator; co-design of MoE expert placement under photonic locality; pipeline-parallel + ZeRO-3 training stage model.